Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Bits and Bytes

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS

What is a Computing System?

A computing system:

- consists of **Software** and **Hardware** working together
- allows **program computations**, i.e., execution of programs

Common examples are your desktop PCs, laptops, smartphones, smartwatches, data centers, supercomputers, smart TVs, smart fridges, and so on!

Computations manipulate data

Well, basically:

- **perform** calculations: a billion calculations per second!
- **remember** results: hundreds of gigabytes of storage!

What kind of **calculations**?

- *built-in* operations included in the programming language
- custom, ad hoc, defined by the programmer

The **essence** of a computation: store, manipulate, and move **data**.

Data in computing means ***bits***

The **bit** is the most basic unit of information in computing. The name is a portmanteau of *binary digit*. The bit represents a *logical state* with one of **two** possible values:

Representation	First value	Second value
Binary digit	1	0
Boolean	true	false
Electronic	on	off

This is how computers see data (pictures, text, everything!):

How can we interpret bits?

Positional Notation

The binary representation is based on a ***positional notation***, which you know from decimal numbers!

For example, if we consider the numbers 3, 43, 543, and 6543:

$$3_{10} = \underbrace{3 \times 10^0}_{\text{first position}}$$

$$43_{10} = \underbrace{4 \times 10^1}_{\text{second position}} + \underbrace{3 \times 10^0}_{\text{first position}}$$

$$543_{10} = \underbrace{5 \times 10^2}_{\text{third position}} + \underbrace{4 \times 10^1}_{\text{second position}} + \underbrace{3 \times 10^0}_{\text{first position}}$$

$$6543_{10} = \underbrace{6 \times 10^3}_{\text{fourth position}} + \underbrace{5 \times 10^2}_{\text{third position}} + \underbrace{4 \times 10^1}_{\text{second position}} + \underbrace{3 \times 10^0}_{\text{first position}}$$

We consider digits based on their position, from the rightmost one to the leftmost one. We then multiply each digit by the base (10 in this case) raised to the power of the position (minus one).

NOTE: The subscript 10 indicates that the number is in base 10.

In other words, we assign a different weight to each digit based on its position:

Digit	**6**	**5**	**4**	**3**
Position	3	2	1	0
Weight	10^3	10^2	10^1	10^0

More generally, we can write for the decimal positional system:

$$\sum_{i=0}^{n-1} d_i \times 10^i = \text{value}_{10}$$

where:

- 10 is the ***base*** used for the powers
- d_i is the digit at position i (from right to left)
- n is the number of digits
- $d_i \in \{0, 1, \ldots, 9\}$

We can generalize our positional system to any base b:

$$\sum_{i=0}^{n-1} d_i \times b^i = \text{value}_{10}$$

where:

- b is the base of the powers
- d_i is the digit at position i (from right to left)
- n is the number of digits
- $d_i \in \{0, 1, \ldots, b-1\}$
- for convenience, the result (value_{10}) is kept in base 10

The base b can be any positive integer, but it is common to use powers of 2, 8, 10, and 16.

Computers work with binary data, so $b = 2$.

Why do computers prefer the binary representation?

Computers are devices that work with electric signals with two state levels:

Why does the signal have only two levels?

It makes it more resistant to noise and interference since it uses extreme, easy-to-identify cases:

- 0: switch off, low signal, no current, or minimum voltage
- 1: switch on, high signal, maximum current, or maximum voltage

Electrical devices can use more levels, but that makes the signal harder to interpret. For instance:

- A light bulb with two states (on or off) makes it easier to identify the state.
- A light bulb with multiple brightness levels needs precise detection: 10% vs 20% brightness? More complex and costly.

Humans and Computers are quite different

What is *intuitive* for humans is not necessarily *intuitive* for computers:

- In the context of computer systems, *intuitive* means *easy to implement*.
- In the context of humans, *intuitive* often means *easy to understand*.

In practice:

- The decimal system is intuitive for humans, but not for computers.
- The binary system is intuitive for computers, but not for humans.

Binary representation

$$\sum_{i=0}^{n-1} d_i \times 2^i = \text{value}_{10}$$

where:

- $b = 2$ is the base of the powers
- d_i is the digit at position i (from right to left)
- n is the number of digits
- $d_i \in \{0, 1\}$
- for convenience, the result (value_{10}) is kept in base 10

The powers of 2 are:

Position:	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Power of two:	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value:	256	128	64	32	16	8	4	2	1

Conversion from binary to decimal

We already know how to do that based on what we said before :)

For instance:

- the binary number 1010:

$$1010_2 = \underbrace{1 \times 2^3}_{\text{fourth position}} + \underbrace{0 \times 2^2}_{\text{third position}} + \underbrace{1 \times 2^1}_{\text{second position}} + \underbrace{0 \times 2^0}_{\text{first position}} = 10_{10}$$

$$1010_2 = \underbrace{1 \times 8}_{\text{fourth position}} + \underbrace{0 \times 4}_{\text{third position}} + \underbrace{1 \times 2}_{\text{second position}} + \underbrace{0 \times 1}_{\text{first position}} = 10_{10}$$

- the binary number 1001:

$$1001_2 = \underbrace{1 \times 2^3}_{\text{fourth position}} + \underbrace{0 \times 2^2}_{\text{third position}} + \underbrace{0 \times 2^1}_{\text{second position}} + \underbrace{1 \times 2^0}_{\text{first position}} = 9_{10}$$

$$1001_2 = \underbrace{1 \times 8}_{\text{fourth position}} + \underbrace{0 \times 4}_{\text{third position}} + \underbrace{0 \times 2}_{\text{second position}} + \underbrace{1 \times 1}_{\text{first position}} = 9_{10}$$

- the binary number 11010:

$$11010_2 = \underbrace{1 \times 2^4}_{\text{fifth position}} + \underbrace{1 \times 2^3}_{\text{fourth position}} + \underbrace{0 \times 2^2}_{\text{third position}} + \underbrace{1 \times 2^1}_{\text{second position}} + \underbrace{0 \times 2^0}_{\text{first position}} = 26_{10}$$

$$11010_2 = \underbrace{1 \times 16}_{\text{fifth position}} + \underbrace{1 \times 8}_{\text{fourth position}} + \underbrace{0 \times 4}_{\text{third position}} + \underbrace{1 \times 2}_{\text{second position}} + \underbrace{0 \times 1}_{\text{first position}} = 26_{10}$$

- the binary number 10001:

$$10001_2 = \underbrace{1 \times 2^4}_{\text{fifth position}} + \underbrace{0 \times 2^3}_{\text{fourth position}} + \underbrace{0 \times 2^2}_{\text{third position}} + \underbrace{0 \times 2^1}_{\text{second position}} + \underbrace{1 \times 2^0}_{\text{first position}} = 17_{10}$$

$$10001_2 = \underbrace{1 \times 16}_{\text{fifth position}} + \underbrace{0 \times 8}_{\text{fourth position}} + \underbrace{0 \times 4}_{\text{third position}} + \underbrace{0 \times 2}_{\text{second position}} + \underbrace{1 \times 1}_{\text{first position}} = 17_{10}$$

First n binary numbers ($n = 16$)

$$
\begin{aligned}
0_2 &= 0_{10} \\
1_2 &= 1_{10} \\
10_2 &= 2_{10} \\
11_2 &= 3_{10} \\
100_2 &= 4_{10} \\
101_2 &= 5_{10} \\
110_2 &= 6_{10} \\
111_2 &= 7_{10} \\
1000_2 &= 8_{10} \\
1001_2 &= 9_{10} \\
1010_2 &= 10_{10} \\
1011_2 &= 11_{10} \\
1100_2 &= 12_{10} \\
1101_2 &= 13_{10} \\
1110_2 &= 14_{10} \\
1111_2 &= 15_{10}
\end{aligned}
$$

Interesting properties of binary numbers

Even numbers end with 0, **odd** numbers end with 1:

$$
\begin{aligned}
0_2 &= 0_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
1_2 &= 1_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
10_2 &= 2_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
11_2 &= 3_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
100_2 &= 4_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
101_2 &= 5_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
110_2 &= 6_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
111_2 &= 7_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
1000_2 &= 8_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
1001_2 &= 9_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
1010_2 &= 10_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
1011_2 &= 11_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
1100_2 &= 12_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
1101_2 &= 13_{10} && \text{last bit is } 1 \Rightarrow \text{odd} \\
1110_2 &= 14_{10} && \text{last bit is } 0 \Rightarrow \text{even} \\
1111_2 &= 15_{10} && \text{last bit is } 1 \Rightarrow \text{odd}
\end{aligned}
$$

Binary numbers composed only of n digits equal to 1 are equal to $2^n - 1$:

$$
\begin{aligned}
1_2 &= 1 \times 2^0 \\
11_2 &= 1 \times 2^1 + 1 \times 2^0 \\
111_2 &= 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
1111_2 &= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
11111_2 &= 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
111111_2 &= 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
1111111_2 &= 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
11111111_2 &= 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0
\end{aligned}
$$

Binary numbers composed only of n digits equal to 0 are equal to 0. This is trivial!

Conversion from decimal to binary

Keep dividing the number by 2 until the quotient is 0, then take the remainders in reverse order.

For instance, when converting 102_{10} to binary:

	quotient	remainder
$102 \div 2 =$	51	0
$51 \div 2 =$	25	1
$25 \div 2 =$	12	1
$12 \div 2 =$	6	0
$6 \div 2 =$	3	0
$3 \div 2 =$	1	1
$1 \div 2 =$	0	1

The result is 1100110_2.

When converting 1056_{10} to binary:

	quotient	remainder
$1056 \div 2$	$= 528$	0
$528 \div 2$	$= 264$	0
$264 \div 2$	$= 132$	0
$132 \div 2$	$= 66$	0
$66 \div 2$	$= 33$	0
$33 \div 2$	$= 16$	1
$16 \div 2$	$= 8$	0
$8 \div 2$	$= 4$	0
$4 \div 2$	$= 2$	0
$2 \div 2$	$= 1$	0
$1 \div 2$	$= 0$	1

The result is 10000100000_2

Why do computers use hexadecimal numbers?

Internally, computers use binary numbers. However, when data must be represented in a human-readable format, there are two common choices:

- decimal:
  - pro:
    - most intuitive and easy for humans
    - more compact (less digits to represent the same number) than binary
  - cons:
    - not compact (more digits to represent the same number) than larger bases
    - hard to convert from binary: we hate divisions!
- hexadecimal:
  - pro:
    - compact (less digits to represent the same number) than binary and decimal
    - easy to convert from binary: we love powers of 2!
  - cons:
    - not intuitive as decimal but not too bad

When data must be shown in raw format, computers show data in hexadecimal.

Hexadecimal numbers

$$\sum_{i=0}^{n-1} d_i \times 16^i = \text{value}_{10}$$

where:

- $b = 16$ is the base of the powers
- d_i is the digit at position i (from right to left)
- n is the number of digits
- $d_i \in \{0, 1, \ldots, 9, A, B, C, D, E, F\}$
- $d_i > 9$ are represented with a single digit: $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$, $F = 15$
- for convenience, the result (value_{10}) is kept in base 10

The powers of 16 are:

Position:	**5**	**4**	**3**	**2**	**1**	**0**
Power of 16:	16^5	16^4	16^3	16^2	16^1	16^0
Value:	1048576	65536	4096	256	16	1

For instance, the number $1A3FA_{16}$ is:

$$1A3FA_{16} = \underbrace{1 \times 16^4}_{\text{fifth position}} + \underbrace{A \times 16^3}_{\text{fourth position}} + \underbrace{3 \times 16^2}_{\text{third position}} + \underbrace{F \times 16^1}_{\text{second position}} + \underbrace{A \times 16^0}_{\text{first position}} = 107514_{10}$$

- $1A3FA_{16}$ is a 5-digit number in base 16.
- $1A3FA_{16}$ is a 6-digit number in base 10: 107514_{10}
- $1A3FA_{16}$ is a 6-digit number in base 2: 11010001111111010_2

The difference in number of digits increases as the number increases.

Conversion from binary to hexadecimal

We can convert a binary number to a hexadecimal number by:

1. grouping the binary digits into sets of four, starting from the right.
2. Each group of four binary digits can be converted to a single hexadecimal digit.

For instance, 1101000111111010_2:

$$1_2\ 1010_2\ 0011_2\ 1111_2\ 1010_2 = \underbrace{1_2}_{1_{16}}\ \underbrace{1010_2}_{A_{16}}\ \underbrace{0011_2}_{3_{16}}\ \underbrace{1111_2}_{F_{16}}\ \underbrace{1010_2}_{A_{16}} = 1A3FA_{16}$$

Conversion from hexadecimal to binary

Opposite strategy:

1. Convert each hexadecimal digit to its binary equivalent.
2. Combine the binary digits to form the final binary number.

For instance, $1A3FA_{16}$:

$$1_{16}\ A_{16}\ 3_{16}\ F_{16}\ A_{16} = \underbrace{1_{16}}_{1_2}\ \underbrace{A_{16}}_{1010_2}\ \underbrace{3_{16}}_{0011_2}\ \underbrace{F_{16}}_{1111_2}\ \underbrace{A_{16}}_{1010_2} = 1101000111111010_2$$

Turning text into bits

One way to represent text as bits is to use the ***ASCII*** (American Standard Code for Information Interchange) encoding:

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

1. Split the text into characters
2. Take each character of your text and consider the hex value of that character from the ASCII table
3. Convert the hex values to binary
4. Concatenate the binary values

For instance, the text "Hello LUISS":

1. Characters: `H`, `e`, `l`, `l`, `o`, ` `, `L`, `U`, `I`, `S`, `S`
2. Hex values: 48_{16}, 65_{16}, $6C_{16}$, $6C_{16}$, $6F_{16}$, 20_{16}, $4C_{16}$, 55_{16}, 49_{16}, 53_{16}, 53_{16}
3. Binary values: 01001000_2, 01100101_2, 01101100_2, 01101100_2, 01101111_2, 00100000_2, 01010101_2, 01001001_2, 01001111_2, 01001111_2
4. Concatenated binary values: $0100100001100101011011000110110001101111001000000100110001010101010010($

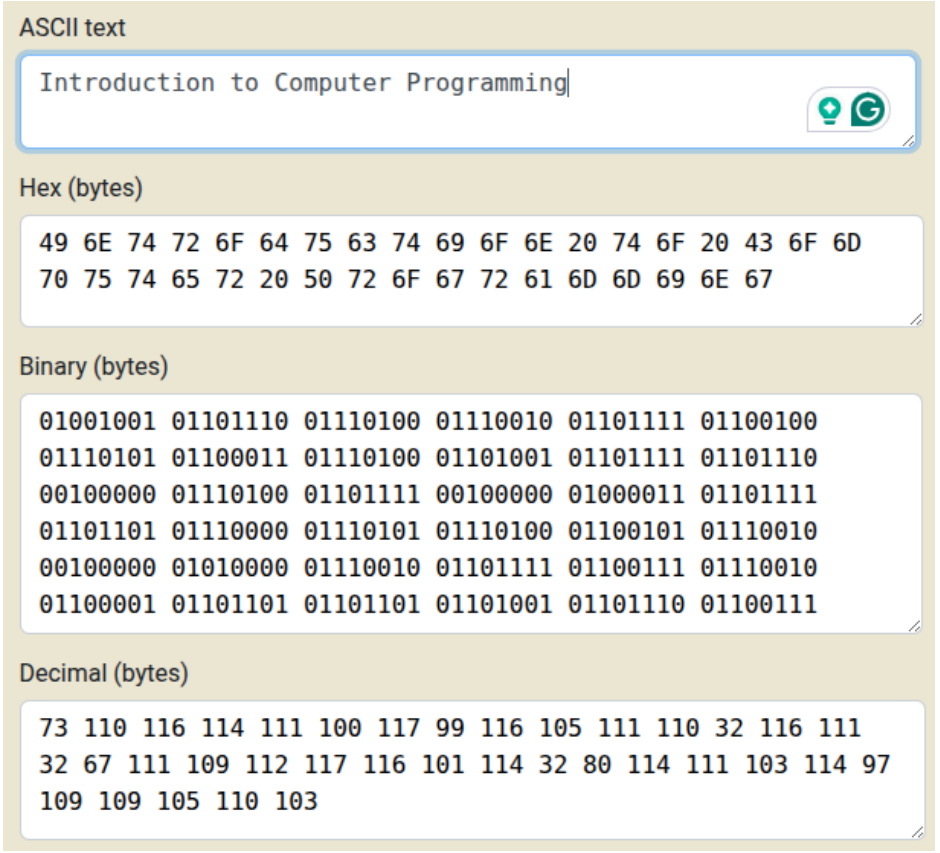
Turning bits to text

1. Split the binary bits into groups of 8
2. Convert the binary values to hex
3. Convert the hex values to characters using the ASCII table
4. Combine the characters to obtain the text

For instance, in the previous example:

1. Grouped binary values:
   $01001000_2\ 01100101_2\ 01101100_2\ 01101100_2\ 01101111_2\ 00100000_2\ 01001100_2\ 0101$
2. Hex values: 48_{16}, 65_{16}, $6C_{16}$, $6C_{16}$, $6F_{16}$, 20_{16}, $4C_{16}$, 55_{16}, 49_{16}, 53_{16}, 53_{16}
3. Characters: `H`, `e`, `l`, `l`, `o`, ` `, `L`, `U`, `I`, `S`, `S`
4. Text: `Hello LUISS`

Tool for the text-to-binary conversion



https://www.rapidtables.com/convert/number/ascii-hex-bin-dec-converter.html

Text is for humans, binary is for machines

Computers are fine with binary and do not need to convert between binary and text.

Computers do convert between binary and text, to help us humans!

Text is shown on the screen for our benefit, but remember that it is stored in binary!

Other data formats

Depending on the specific data type, there is a **standardized** way of turning a sequence of bits into a human-readable format.

For instance, a simple yet inefficient image format:

- split bits into groups of 8
- each group is a number between 0 and 255
- each number is seen as a different pixel
- each pixel value is displayed on the screen with a different color
- there are only 256 colors available

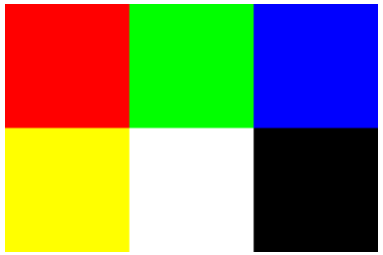
This is a very inefficient way of representing an image, but it is simple to understand. Image formats, such as JPEG or PNG, are much more efficient and effective:

- they can use more than 256 colors... millions of colors!
- they can use compression to reduce the size of the file: if some adjacent pixels have the same color, they can be represented in a more compact way!

PPM Image format

This is an example of a color RGB image stored in PPM format:

```
P3
# "P3" means this is a RGB color image in ASCII
# "3 2" is the width and height of the image in pixels
# "255" is the maximum value for each color
# This, up through the "255" line below, is the header.
# Everything after that is the image data: RGB triplets.
# In order: red, green, blue, yellow, white, and black.
3 2
255
255   0   0
  0 255   0
  0   0 255
255 255   0
255 255 255
  0   0   0
```



Wikipedia > PPM format

Byte

A **byte** is a unit of information made up of **8 bits**. Most systems and data formats are designed to process data in bytes, the most common unit when dealing with sequences of bits. Indeed, in computer science, bytes are often used to measure the amount of data stored or transmitted.

Notice that:

- 8 bits can represent 256 different values (from 0 to 255)
- 8 bits can be written as 2 hex digits

For instance, 1110110011_2:

$$\underbrace{10110011_2}_{\text{8 bits}} = \underbrace{1011_2\,0011_2}_{\text{1 byte}} = \underbrace{B_{16}3_{16}}_{\text{2 hex digits}} = 187_{10}$$

For values larger than 255, we still group by 8 bits, ending with more than one byte. When the number of bits is not a multiple of 8, we add leading zeros to make it a multiple of 8.

For instance:

$$\underbrace{1110110011_2}_{\text{10 bits}} = \underbrace{0000_2\,0011_2\;1011_2\,0011_2}_{\text{2 bytes}} = \underbrace{0_{16}\,3_{16}\,B_{16}\,3_{16}}_{\text{4 hex digits}} = 947_{10}$$

Multiple-byte units

Decimal			Binary				
Value	Metric		Value	IEC		Memory	
1000	kB	kilobyte	1024	KiB	kibibyte	KB	kilobyte
1000^2	MB	megabyte	1024^2	MiB	mebibyte	MB	megaby
1000^3	GB	gigabyte	1024^3	GiB	gibibyte	GB	gigabyt
1000^4	TB	terabyte	1024^4	TiB	tebibyte	TB	terabyte
1000^5	PB	petabyte	1024^5	PiB	pebibyte		—
1000^6	EB	exabyte	1024^6	EiB	exbibyte		—
1000^7	ZB	zettabyte	1024^7	ZiB	zebibyte		—
1000^8	YB	yottabyte	1024^8	YiB	yobibyte		—
1000^9	RB	ronnabyte	1024^9	RiB	robibyte		—
1000^{10}	QB	quettabyte	1024^{10}	QiB	quebibyte		—

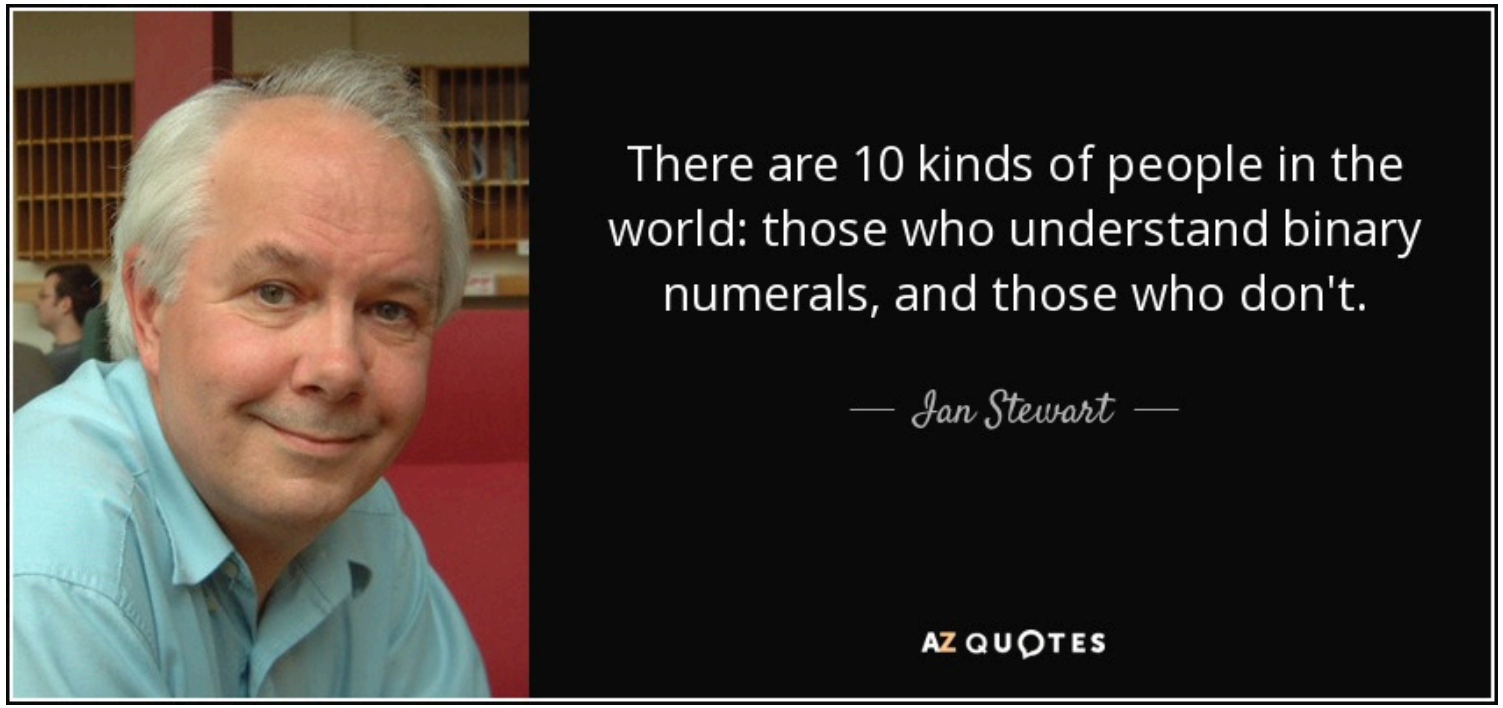
Understanding The Growing World Of Bytes

An interesting infographic that shows the evolution (and scale) of the world's data over time:

Source: Understanding The Growing World Of Bytes

Fun time

Dad's joke time



If you do not get the joke: `10` in binary is 2 :)

Wait a minute...

Check out your hands... What do you see?

I see ten bits! Each finger is either up (`1`) or down (`0`).

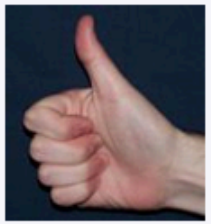
Finger Binary

https://en.wikipedia.org/wiki/Finger_binary

	Left hand					Right hand				
	Thumb	Index	Middle	Ring	Pinky	Pinky	Ring	Middle	Index	Thumb
Power of two	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	512	256	128	64	32	16	8	4	2	1

Using two hands we can count up to 1023, that is $2^{10} - 1$!

0 = empty sum

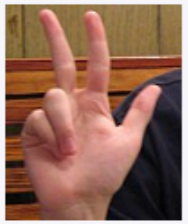


1 = 1

2 = 2

4 = 4

6 = 4 + 2



7 = 4 + 2 + 1

14 = 8 + 4 + 2

16 = 16

19 = 16 + 2 + 1

26 = 16 + 8 + 2

28 = 16 + 8 + 4

30 = 16 + 8 + 4 + 2

31 = 16 + 8 + 4 + 2 + 1